

计算几何

向量
内积
外积
混合积
弧度制与坐标系
几何关系
几何基础
Pick定理
凸包
半平面交
最小圆覆盖
旋转卡壳
三角剖分
平面最近点对

计算几何

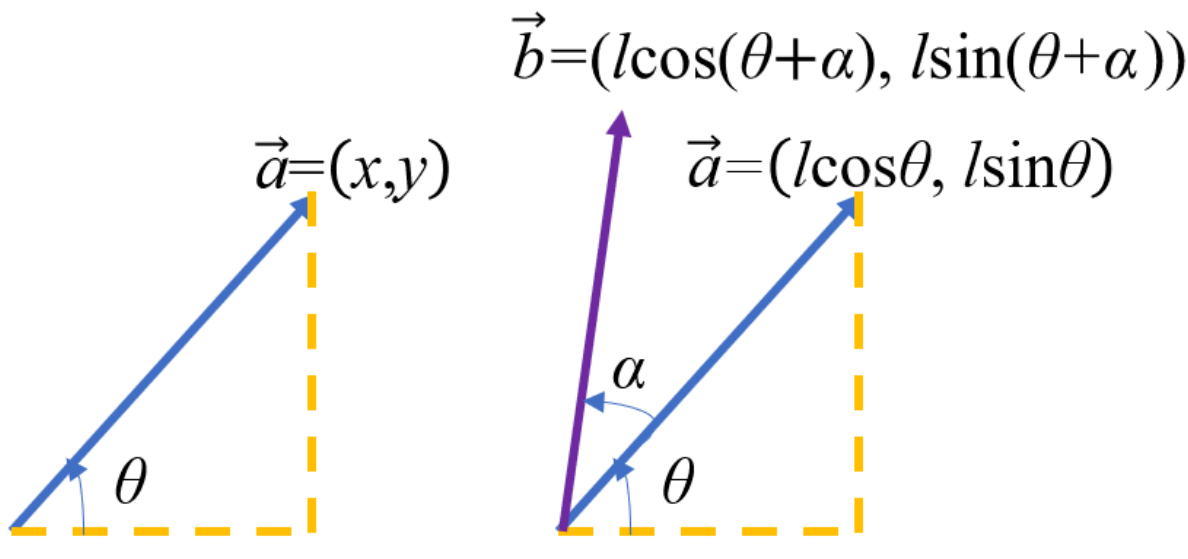
向量

三点共线: A, B, C 三点共线 $\Rightarrow \overrightarrow{OB} = \lambda \overrightarrow{OA} + (1 - \lambda) \overrightarrow{OC}$

定比分点: 在三角形 ABC 中, 若 D 为 BC 的 n 等分点 ($n BD = k DC$) $\Rightarrow \overrightarrow{AD} = \frac{n}{k+n} \overrightarrow{AB} + \frac{k}{k+n} \overrightarrow{AC}$

共面向量基本定理: 若存在俩不共线的向量 $\mathbf{x}, \mathbf{y}$, 则向量 $\mathbf{p}$ 与 $\mathbf{x}, \mathbf{y}$ 共面 $\Leftrightarrow \exists (a, b) \in \mathbb{R}^2, s. t. \mathbf{p} = a\mathbf{x} + b\mathbf{y}$

向量旋转: $\mathbf{a} = (x, y)$, 倾角 θ , 长度为 $l = \sqrt{x^2 + y^2}$, 则 $x = l \cos \theta, y = l \sin \theta$, 其逆时针旋转 α 度, 得 $\mathbf{b} = (l \cos(\theta + \alpha), l \sin(\theta + \alpha))$



由三角恒等变换得, $\mathbf{b} = (l(\cos \theta \cos \alpha - \sin \theta \sin \alpha), l(\sin \theta \cos \alpha + \cos \theta \sin \alpha))$, 化简, $\mathbf{b} = (l \cos \theta \cos \alpha - l \sin \theta \sin \alpha, l \sin \theta \cos \alpha + l \cos \theta \sin \alpha)$

把上面的 x, y 代回来得 $\mathbf{b} = (x \cos \alpha - y \sin \alpha, y \cos \alpha + x \sin \alpha) = (x, y) \begin{bmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{bmatrix}$

向量与矩阵: 向量与矩阵运算相似, 考虑将向量写成矩阵形式, 将向量问题转化为矩阵问题

内积

(i) **定义:** $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos \theta$

(ii) **几何意义:** $\mathbf{a} \cdot \mathbf{b} = \mathbf{a}$ 的模与 $\mathbf{b}$ 在 $\mathbf{a}$ 的方向上的投影的乘积

(iii) **计算:** 两个向量 $\mathbf{a} = (x_1, y_1, z_1), \mathbf{b} = (x_2, y_2, z_2)$ 内积的结果是一个数量 $\mathbf{a} \cdot \mathbf{b} = x_1 x_2 + y_1 y_2 + z_1 z_2$

(iv) 交换律: $\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a}$

外积

(i) 定义: 记为 $\mathbf{a} \times \mathbf{b}$, 其模与方向定义如下:

1. $|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}||\mathbf{b}|\sin\langle \mathbf{a}, \mathbf{b} \rangle$
2. $\mathbf{a} \times \mathbf{b}$ 与 $\mathbf{a}, \mathbf{b}$ 都垂直, 且 $\mathbf{a}, \mathbf{b}, \mathbf{a} \times \mathbf{b}$ 符合右手法则

(ii) 几何意义: $|\mathbf{a} \times \mathbf{b}|$ 是以 $\mathbf{a}, \mathbf{b}$ 为邻边的平行四边形的面积, 顺负逆正

(iii) 计算: 两个向量 $\mathbf{a} = (x_1, y_1, z_1), \mathbf{b} = (x_2, y_2, z_2)$ 外积的结果是一个向量 $\mathbf{c}$ 记作 $\mathbf{c} = \mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \end{vmatrix}$

其中 $\mathbf{i}, \mathbf{j}, \mathbf{k}$ 表示和坐标轴 x, y, z 平行的单位向量, 并写在对应坐标处。展开得 $\mathbf{c} = (y_1z_2 - y_2z_1, x_2z_1 - x_1z_2, x_1y_2 - x_2y_1)$

(iv) 反交换律: $\mathbf{a} \times \mathbf{b} = -\mathbf{b} \times \mathbf{a}$

(v) 恒等式: $(\mathbf{a} \times \mathbf{b})^2 = a^2b^2 - (\mathbf{a} \cdot \mathbf{b})^2$

(vi) 二重外积:

1. 特殊情况: $(\mathbf{a} \times \mathbf{b}) \times \mathbf{a} = (a^2)\mathbf{b} - (\mathbf{a}\mathbf{b})\mathbf{a}$

证明: $\mathbf{a} \times \mathbf{b}$ 与 $\mathbf{a}$ 和 $\mathbf{b}$ 垂直, 且等式左端与 $\mathbf{a} \times \mathbf{b}$ 垂直 $\Rightarrow$ 等式左端与 $\mathbf{a}$ 和 $\mathbf{b}$ 共面

因此设 $(\mathbf{a} \times \mathbf{b}) \times \mathbf{a} = \lambda\mathbf{a} + \mu\mathbf{b}$, 同时对 $\mathbf{a}$ 和 $\mathbf{b}$ 分别做内积得

$$\lambda a^2 + \mu(\mathbf{a}\mathbf{b}) = 0, \quad \lambda(\mathbf{a}\mathbf{b}) + \mu b^2 = (\mathbf{b}, \mathbf{a} \times \mathbf{b}, \mathbf{a}) = (\mathbf{a} \times \mathbf{b})^2$$

$$\text{又由 } (\mathbf{a} \times \mathbf{b})^2 = a^2b^2 - (\mathbf{a}\mathbf{b})^2 \Rightarrow \lambda = -\mathbf{a}\mathbf{b}, \mu = a^2$$

2. 普遍情况: $(\mathbf{a} \times \mathbf{b}) \times \mathbf{c} = (\mathbf{a}\mathbf{c})\mathbf{b} - (\mathbf{b}\mathbf{c})\mathbf{a}$

证明: 这里只需考虑三个向量均为非零且不共线的情况, 其他特例为显然的

$\mathbf{a}, \mathbf{b}$ 和 $\mathbf{a} \times \mathbf{b}$ 不共面, 因此设 $\mathbf{c} = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma(\mathbf{a} \times \mathbf{b})$

$$\Rightarrow (\mathbf{a} \times \mathbf{b}) \times \mathbf{c} = (\mathbf{a} \times \mathbf{b}) \times (\alpha\mathbf{a} + \beta\mathbf{b} + \gamma(\mathbf{a} \times \mathbf{b})) = \alpha(\mathbf{a} \times \mathbf{b}) \times \mathbf{a} + \beta(\mathbf{a} \times \mathbf{b}) \times \mathbf{b}$$

$$\text{由上文 } (\mathbf{a} \times \mathbf{b}) \times \mathbf{a} = (a^2)\mathbf{b} - (\mathbf{a}\mathbf{b})\mathbf{a}, (\mathbf{a} \times \mathbf{b}) \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a}) \times \mathbf{b} = -(\mathbf{b}^2)\mathbf{a} + (\mathbf{a}\mathbf{b})\mathbf{b}$$

$$\Rightarrow (\mathbf{a} \times \mathbf{b}) \times \mathbf{c} = \alpha((a^2)\mathbf{b} - (\mathbf{a}\mathbf{b})\mathbf{a}) + \beta((\mathbf{a}\mathbf{b})\mathbf{b} - (\mathbf{b}^2)\mathbf{a}) = (\alpha(-\mathbf{a}\mathbf{b}) + \beta(-\mathbf{b}^2))\mathbf{a} + (\alpha a^2 + \beta \mathbf{a}\mathbf{b})\mathbf{b} = (\mathbf{a}\mathbf{c})\mathbf{b} - (\mathbf{b}\mathbf{c})\mathbf{a}$$

3. 反交换性: $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = (\mathbf{a}\mathbf{c})\mathbf{b} - (\mathbf{a}\mathbf{b})\mathbf{c}$

4. 拉格朗日恒等式: $(\mathbf{a} \times \mathbf{b})(\mathbf{c} \times \mathbf{d}) = (\mathbf{a}\mathbf{c})(\mathbf{b}\mathbf{d}) - (\mathbf{a}\mathbf{d})(\mathbf{b}\mathbf{c})$

$$\text{证明: } (\mathbf{a} \times \mathbf{b})(\mathbf{c} \times \mathbf{d}) = (\mathbf{c}, \mathbf{d}, \mathbf{a} \times \mathbf{b}) = (\mathbf{a} \times \mathbf{b}, \mathbf{c}, \mathbf{d}) = ((\mathbf{a} \times \mathbf{b}) \times \mathbf{c})\mathbf{d} = (\mathbf{b}(\mathbf{a}\mathbf{c}) - \mathbf{a}(\mathbf{b}\mathbf{c}))\mathbf{d} = (\mathbf{a}\mathbf{c})(\mathbf{b}\mathbf{d}) - (\mathbf{a}\mathbf{d})(\mathbf{b}\mathbf{c})$$

可见, 前文的恒等式 $(\mathbf{a} \times \mathbf{b})^2 = a^2b^2 - (\mathbf{a}\mathbf{b})^2$ 是拉格朗日的恒等式的特殊情形

混合积

(i) 定义: 设 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 是空间中的三个向量 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 的混合积, 记为 $[\mathbf{a}\mathbf{b}\mathbf{c}]$ 或 $(\mathbf{a}, \mathbf{b}, \mathbf{c})$ 或 $(\mathbf{a}\mathbf{b}\mathbf{c})$

(ii) 几何意义: 其绝对值 $|(\mathbf{a} \times \mathbf{b})\mathbf{c}|$ 的几何意义表示以 $\mathbf{a}, \mathbf{b}, \mathbf{c}$ 为棱的平行四边形的体积

(iii) 计算: 向量的混合积 $(\mathbf{a}, \mathbf{b}, \mathbf{c}) = (\mathbf{a} \times \mathbf{b})\mathbf{c} = \begin{vmatrix} a_x & b_x & c_x \\ a_y & b_y & c_y \\ a_z & b_z & c_z \end{vmatrix} = a_x b_y c_z + a_y b_z c_x + a_z b_x c_y - a_z b_y c_x - a_y b_x c_z - a_x b_z c_y$

向量的混合积可以用来计算四面体的体积: $V = \frac{1}{6} |\vec{AB}, \vec{AC}, \vec{AD}|$

$(\mathbf{a}, \mathbf{b}, \mathbf{c})$ 的符号是正还是负, 取决于 $\mathbf{a} \times \mathbf{b}$ 与 $\mathbf{c}$ 的夹角是锐角还是钝角

(iv) 轮换对称性: $(\mathbf{a}, \mathbf{b}, \mathbf{c}) = (\mathbf{b}, \mathbf{c}, \mathbf{a}) = (\mathbf{c}, \mathbf{a}, \mathbf{b}) = -(\mathbf{b}, \mathbf{a}, \mathbf{c}) = -(\mathbf{a}, \mathbf{c}, \mathbf{b}) = -(\mathbf{c}, \mathbf{b}, \mathbf{a}), (\mathbf{a} \times \mathbf{b})\mathbf{c} = \mathbf{a}(\mathbf{b} \times \mathbf{c})$

几何关系

俩向量垂直: $a \perp b \Leftrightarrow a \cdot b = 0$

判断俩向量共线: $a = \lambda b \Leftrightarrow |a \cdot b| = |a||b| \Leftrightarrow |a \times b| = 0$

三点共线: 三点 A, B, C 共线 $\Leftrightarrow \overrightarrow{AB}$ 与 $\overrightarrow{BC}$ 共线

俩向量的夹角: $\cos \angle(a, b) = \frac{|a \cdot b|}{|a||b|}$

几何基础

二维几何基础:

图形的记录:

点: 在平面直角坐标系记录 (x, y) , 在极坐标下, 记录 (ρ, φ)

向量: 像点一样记录向量

直线、射线和线段: 记录起点和方向向量

一般式: $Ax + By + C = 0$

斜截式: $y = kx + b$

截距式: $\frac{x}{a} + \frac{y}{b} = 1$

多边形: 按顺(逆)时针记录多边形的每个顶点, 特殊的, 若矩形的各边与坐标轴平行, 只需记录左下角和右上角

曲线: 记录解析式, 对于圆记录半径与圆心, 椭圆可通过坐标轴变换变成圆来记录

基本公式: $\forall \triangle ABC$, 角 A, B, C 对应的边为 a, b, c

正弦定理: $\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$, R 为 $\triangle ABC$ 外接圆半径

余弦定理: $a^2 = b^2 + c^2 - 2bc \cos A$

基本操作:

向量绕起点逆时针旋转: $\forall$ 向量 $a = (x, y)$, 绕起点旋转 θ 度后 $a' = (x, y) \begin{bmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{bmatrix}$

判断一个点在直线的哪边: $\forall$ 点 P , P 在以起点为 Q , 方向向量为 v 的直线的左边 $\Leftrightarrow v \times \overrightarrow{QP} > 0$ (等于 0 时在直线上)

快速排斥实验与跨立实验:

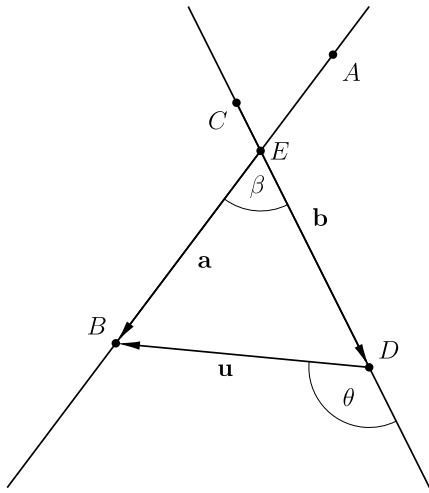
快速排斥: 如果俩条线段没有公共区域 $\Rightarrow$ 一定不相交

跨立实验: 对于一条线段, 另一条线段的俩端在直线的两侧

注: 俩条线段共线但不相交时也可以过跨立实验

一条直线是否交于线段: 跨立实验

俩条直线交点: 若 AB, CD 不平行, 记交点为 E



由上图可知, $|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}||\mathbf{b}| \sin \beta$, $|\mathbf{u} \times \mathbf{b}| = |\mathbf{u}||\mathbf{b}| \sin \theta$

做商得 $T = \frac{|\mathbf{u} \times \mathbf{b}|}{|\mathbf{a} \times \mathbf{b}|} = \frac{|\mathbf{u}| \sin \theta}{|\mathbf{a}| \sin \beta}$

用正弦定理 $\left| \frac{|\mathbf{u}| \sin \theta}{\sin \beta} \right| = l$, l 为 B 距离 E 的距离, 于是只要 $B - T\mathbf{a} = E$

多边形面积:

(i) **叉积:** $S = \frac{\text{叉积}}{2}$

(ii) **海伦公式:** $p = \frac{(a+b+c)}{2}$, $s = \sqrt{p(p-a) * (p-b) * (p-c)}$

三角形四心:

(i) **外心, 外接圆圆心:** 三条中垂线交点, 到三个顶点距离相等

(ii) **内心, 内切圆圆心:** 角平分线交点, 到三边距离相等

(iii) **垂心:** 三条垂线交点

(iv) **重心:** 三条中线交点, 重心在中线上比例 2 : 1, 到三角形三顶点距离的平方和最小, 三角形内到三边距离之积最大

判断一点是否在任意多边形内部:

光线投射算法: 先判断点以端点引一条射线与多边形的交点个数, 奇内偶外

随机取这条射线所在直线的斜率, 为无理数以避免出现射线与多边形某边重合的情况

也可使用多边形中最后一个点作为射线上一点, 这样如果出现射线过多边形某边或某顶点时

可以规定射线经过的点同在射线一侧, 进而做跨立实验即可

判断一点是否在凸多边形内部: 只需判断点是否在所有边的左边(逆时针存储多边形)

判断多边形是否为凸包: 先满足凸包左旋或右旋, 然后看极角趋势变化次数, 小于等于 2 的为凸包

求多边形周长: 直接计算

求多边形面积: 多边形上逆时针点 $p_1, p_2, \dots, p_n$, 任选一点 O , 记 $\mathbf{v}_i = p_i - O$, 那么 $S = \frac{1}{2} \left| \sum_{i=1}^n \mathbf{v}_i \times \mathbf{v}_{(i \bmod n)+1} \right|$

圆与直线相关:

求直线与圆的交点: 先判断直线与圆的关系, 若相离无交点, 相切利用切线算出切点与半径的直线, 转化为求俩直线交点

若与圆交点, 用同样方法求出中点, 再沿直线方向加上半弦长

求两圆交点: 先判断俩圆的位置关系, 若外离或内含则无交点, 相切可以算出俩圆心连线的方向向量积, 利用俩圆半径计算平移距离

若相交与俩点,将一圆圆心与交点连线,用余弦定理算出俩圆心连线与该连线的所成角

将俩圆心连线旋转这个角度得到方向向量,最后平移半径长度即可

极角排序:

使用极角: 使用 $\arctan(y, x)$ 函数, 值为 $(-\pi, \pi]$, 第四象限的极角会比第一象限的小

使用叉乘: 只依靠叉乘不能排序, 不满足偏序关系, 若定义 $\mathbf{x} \times \mathbf{y} > 0$ 则 $\theta_{\mathbf{x}} < \theta_{\mathbf{y}}$

会发现不断在坐标轴上旋转, 一个向量的极角最终会小于自己, 但在同一象限上可以, 所以先比较象限再做叉乘

```
1  template <class T> struct point {    // 使用 f64 可能需要重载比较运算符
2      T x, y;
3      static constexpr T pi = acos(T(-1));
4      point(T x = {}, T y = {}) : x(x), y(y) {}
5      template <class U> operator point<U>() const { return point<U>(U(x), U(y)); }
6
7      point operator - () const { return point(-x, -y); }
8      T operator * (const point & t) const { return x * t.y - y * t.x; }    // 叉积
9      T operator / (const point & t) const { return x * t.x + y * t.y; }    // 内积
10     point operator + (const point & t) const { return point(*this) += t; }
11     point operator - (const point & t) const { return point(*this) -= t; }
12     point operator * (const T & t) const { return point(*this) *= t; }
13     point operator / (const T & t) const { return point(*this) /= t; }
14     bool operator == (const point & t) const { return x == t.x && y == t.y; }
15     bool operator != (const point & t) const { return !(*this == t); }
16     point & operator += (const point & t) { return x += t.x, y += t.y, *this; }
17     point & operator -= (const point & t) { return x -= t.x, y -= t.y, *this; }
18     point & operator *= (const T & t) { return x *= t, y *= t, *this; }
19     point & operator /= (const T & t) { return x /= t, y /= t, *this; }
20     friend istream & operator >> (istream & is, point & t) { return is >> t.x >> t.y; }
21     friend ostream & operator << (ostream & os, point t) { return os << "(" << t.x << ", " << t.y << ")"; }
22     T square() const { return *this / *this; }
23     T len() const { return sqrt(square()); }
24     // 以下均绕 c = (0, 0) 可使用 (p - c) 进行偏移操作
25     int sgn() const { return y > 0 || (y == 0 && x > 0) ? 1 : -1; }
26     T theta(T t = 0) const { return ((t = atan2(y, x)) < 0) * 2 * pi + t; }    // 极角
27     int less(const point & t) const { return sgn() != t.sgn() ? sgn() == 1 : t * (*this) < 0; }    // 极角比较
28     point rotat() const { return point(-y, x); }    // 旋转 pi / 2
29     point rotat(const T & t) const { return point(x * cos(t) - y * sin(t), y * cos(t) + x * sin(t)); }    //
    向量旋转
30
31     struct line {
32         point a, b;    // a -> b
33         line(point a = {}, point b = {}) : a(a), b(b) {}
34
35         point v() const { return b - a; }
36         T P_pos(const point & t) const { return (t - a) * v(); }    // < 0 点在线左侧, == 0 在线上, pos > 0 在右侧
37         T L_pos(const line & t) const { return t.v() * v(); }    // < 0 表示共点时, t 在线左侧, == 0 俩线平行, > 0
    在线右侧
38         T P_dir(const point & t) const { return (t - a) / v(); }    // 可用于投影
39         T L_dir(const line & t) const { return t.v() / v(); }    // < 0 表示反向, == 0 垂直, > 0 同向
40         point L_inter(const line & t) const { return a + v() * (t.P_pos(a) / L_pos(t)); }    // 算俩直线交点, 用前
    判断直线平行或重合
41         int P_on_seg(const point & t) const { return P_pos(t) == 0 && (a - t) / (b - t) <= 0; }    // 判断点是
    否在线段上
42         T P_dis_L(const point & t) const { return v().square() == 0 ? (t - a).len() : abs(P_pos(t)) /
    v().len(); }    // 点到直线距离
43         T P_dis_seg(const point & t) const {    // 点到线段距离
44             if (P_dir(t) < 0) return (t - a).len();
45             if (line(b, a).P_dir(t) < 0) return (t - b).len();
46             return P_dis_L(t);
47         }
48         T seg_dis_seg(const line & t) const {    // 线段到线段距离
49             get<0>(seg_inter(t)) != 0 ? T(0) : min({P_dis_seg(t.a), P_dis_seg(t.b), t.P_dis_seg(a),
    t.P_dis_seg(b)});
50         }
51         point P_proj(const point & t) const { return a + v() * (v() / (t - a) / v().square()); }    // 求点在直
    线上的射影, 即垂足
```

```

52     tuple<int, point, point> seg_inter(const line & t) const { // 判断俩线段是否相交, 0 表示不相交, 1 表示严
    格相交, 2 表示覆盖, 3 表示交于端点
53         if (max(a.x, b.x) < min(t.a.x, t.b.x) || min(a.x, b.x) > max(t.a.x, t.b.x) // 快速排斥
54             || max(a.y, b.y) < min(t.a.y, t.b.y) || min(a.y, b.y) > max(t.a.y, t.b.y)) return {0, {}, {}};
55         if (L_pos(t) == 0) { // 平行
56             if (P_pos(t.a) != 0) return {0, {}, {}}; // 不共线
57             point p1(max(min(a.x, b.x), min(t.a.x, t.b.x)), max(min(a.y, b.y), min(t.a.y, t.b.y)));
58             point p2(min(max(a.x, b.x), max(t.a.x, t.b.x)), min(max(a.y, b.y), max(t.a.y, t.b.y)));
59             if (!P_on_seg(p1)) swap(p1.y, p2.y);
60             if (p1 == p2) return {3, p1, p2};
61             return {2, p1, p2};
62         }
63         auto cp1 = (t.a - a) * (t.b - a), cp2 = (t.a - b) * (t.b - b); // 跨立试验
64         auto cp3 = (a - t.a) * (b - t.a), cp4 = (a - t.b) * (b - t.b);
65         if ((cp1 > 0 && cp2 > 0) || (cp1 < 0 && cp2 < 0) || (cp3 > 0 && cp4 > 0) || (cp3 < 0 && cp4 <
0)) return {0, {}, {}};
66         point p = L_inter(t);
67         if (cp1 != 0 && cp2 != 0 && cp3 != 0 && cp4 != 0) return {1, p, p};
68         return {3, p, p};
69     }
70     line mid_L() const { return line((a + b) / 2, (a + b) / 2 + (b - a).rotat()); }
71 };
72
73 struct circle {
74     point o;
75     T r;
76     circle(point o = {}, T r = {}) : o(o), r(r) {}
77     circle(point a, point b, point c) : o(line(a, b).mid_L().L_inter(line(b, c).mid_L())), r((a -
o).len()) {}
78
79     deque<line> P_tan(const point & t) const { // 过点做圆的切线, 返回切线
80         auto v = o - t;
81         if (v.square() < r * r) return {}; // 点在圆内
82         if (v.square() == r * r) return {{t, t + v.rotat()}}; // 点在圆上
83         T ang = asin(r / v.len());
84         return {{t, t + v.rotat(-ang)}, {t, t + v.rotat(ang)}};
85     }
86     pair<int, deque<line>> C_tan(const circle & t, deque<line> res = {}) const {
87         auto v = r < t.r ? o - t.o : t.o - o; // 大的指向小的
88         T d2 = v.square(), dr = abs(t.r - r), sr = t.r + r;
89         if (d2 == 0 && r == t.r) return {-1, {}}; // 重合, 无数条切线
90         if (d2 < dr * dr) return {0, {}}; // 内含, 无切线
91         if (d2 == dr * dr) return {1, {{o + v * r / v.len(), o + v * r / v.len() + v.rotat()}}}; //
    内切
92         T ang = acos(dr / v.len());
93         res.emplace_back(o + v.rotat(ang) * r / v.len(), t.o + v.rotat(ang) * t.r / v.len());
94         res.emplace_back(o + v.rotat(-ang) * r / v.len(), t.o + v.rotat(-ang) * t.r / v.len());
95         if (d2 == sr * sr) {
96             res.emplace_back(o + v * r / v.len(), o + v * r / v.len() + v.rotat());
97             return {3, res};
98         } else if (d2 > sr * sr) {
99             ang = acos(sr / v.len());
100             res.emplace_back(o + v.rotat(ang) * r / v.len(), t.o + (-v).rotat(ang) * t.r / v.len());
101             res.emplace_back(o + v.rotat(-ang) * r / v.len(), t.o + (-v).rotat(-ang) * t.r / v.len());
102             return {4, res};
103         }
104         return {2, res};
105     }
106     deque<point> l_inter(const line & t) const { // 直线与圆交点
107         auto v = t.v(), p = t.P_proj(o);
108         T d = (o - p).len();
109         if (d > r) return {}; // 直线与圆不相交
110         if (d == r) return {p}; // 直线与圆相切
111         T l = sqrt(r * r - d * d);
112         return {p + v * l / v.len(), p - v * l / v.len()};
113     }
114     deque<point> C_inter(const circle & t) const { // 俩圆交点
115         auto v = t.o - o;
116         T d = v.len();
117         if (d > r + t.r || d < abs(r - t.r)) return {}; // 相离|内含

```

```

118         T a = (v.square() + r * r - t.r * t.r) / (2 * d), h = sqrt(r * r - a * a);
119         auto v1 = v * a / d, vh = v.rotat() * h / d;
120         return {o + v1 + vh, o + v1 - vh};
121     }
122 };
123
124     static auto hull(const vector<point> & p) { // 下标从 0, 逆时针
125         deque<int> id(p.size()), h, l; iota(id.begin(), id.end(), 0);
126         sort(id.begin(), id.end(), [&](auto & a, auto & b) { return p[a].x != p[b].x ? p[a].x < p[b].x :
p[a].y < p[b].y; });
127         for (int i = 0; i < p.size(); i++) if (!i || p[id[i - 1]] != p[id[i]]) { // 下凸壳和上凸壳
128             for ( ; h.size() > 1 && line(p[h.back()], p[h[h.size() - 2]]).P_pos(p[id[i]]) <= 0;
h.pop_back());
129             for ( ; l.size() > 1 && line(p[l[0]], p[l[1]]).P_pos(p[id[i]]) >= 0; l.pop_front());
130             h.emplace_back(id[i]), l.emplace_front(id[i]);
131         }
132         if (l.size() > 1) h.insert(h.end(), l.begin() + 1, l.end() - 1);
133         return move(h);
134     }
135
136     static auto hull_diameter(const vector<point> & p, deque<int> res = {}) { // 返回凸包直径的两点
137         if (res.resize(max<int>(2, p.size())), p.size() < 3) return res;
138         for (int i = 0, j = p.size() - 1, k = 1; i < p.size(); j = i++) {
139             for ( ; abs(line(p[i], p[j]).P_pos(p[k])) < abs(line(p[i], p[j]).P_pos(p[(k + 1) % p.size()]));
(k += 1) %= p.size());
140             res = max({res, {i, k}, {j, k}}, [&](auto & a, auto & b) { return (p[a[0]] - p[a[1]]).square() <
(p[b[0]] - p[b[1]]).square(); });
141         }
142         return move(res);
143     }
144
145     static auto P_In_Polygon(const vector<point> & p, const point & t, int cnt = 0) {
146         for (int i = 0, j = p.size() - 1; i < p.size(); j = i++) {
147             if (line(p[j], p[i]).P_on_seg(t)) return cnt = 1;
148             if (p[j].x < t.x && p[i].x >= t.x && line(p[j], p[i]).P_pos(t) > 0) cnt ^= 1;
149             if (p[j].x >= t.x && p[i].x < t.x && line(p[j], p[i]).P_pos(t) < 0) cnt ^= 1;
150         }
151         return cnt;
152     }
153
154     static auto seg_In_Polygon(const vector<point> & p, const line & t) {
155         if (!P_In_Polygon(p, t.a) || !P_In_Polygon(p, t.b)) return 0;
156         for (int i = 0, j = p.size() - 1, k = p.size() - 2; i < p.size(); k = j, j = i++) {
157             auto now = line(p[k], p[j]);
158             auto [st, p1, p2] = now.seg_inter(t);
159             if (st == 1) return 0;
160             if (st == 0) continue;
161             if (st == 2) {
162                 if (t.P_on_seg(p[j]) && p[j] != t.a && p[j] != t.b && (p[j] - p[k]) * (p[i] - p[j]) > 0)
return 0;
163             } else {
164                 if (p1 != p[k] && p1 != p[j]) {
165                     if (now.P_pos(t.a) > 0 || now.P_pos(t.b) > 0) return 0;
166                 } else if (p1 == p[j]) {
167                     if (t.a == p[j]) {
168                         if (t.P_pos(p[k]) < 0) {
169                             if (t.P_pos(p[i]) < 0 && now.P_pos(p[i]) < 0) return 0;
170                         } else {
171                             if (t.P_pos(p[i]) < 0 || now.P_pos(p[i]) < 0) return 0;
172                         }
173                     } else if (t.b == p[j]) {
174                         if (t.P_pos(p[k]) > 0) {
175                             if (t.P_pos(p[i]) > 0 && now.P_pos(p[i]) < 0) return 0;
176                         } else {
177                             if (t.P_pos(p[i]) > 0 || now.P_pos(p[i]) < 0) return 0;
178                         }
179                     } else {
180                         if (t.P_pos(p[k]) < 0) {
181                             if (t.P_pos(p[i]) > 0 || now.P_pos(p[i]) < 0) return 0;
182                         } else {

```

```

183         if (t.P_pos(p[i]) < 0 || now.P_pos(p[i]) < 0) return 0;
184     }
185 }
186 }
187 }
188 }
189 return 1;
190 }
191
192 static auto half_plane(const vector<line> & h, deque<int> q = {}, deque<point> t = {}) {
193     vector<int> p(h.size()); iota(p.begin(), p.end(), 0); // 下标从 0, 逆时针, 左侧的半平面交, 使用时一定 TMD
194     加上外边框, 最好是矩形的
195     sort(p.begin(), p.end(), [&](int x, int y) { return h[x].v().less(h[y].v()); });
196     q.emplace_back(p[0]);
197     for (int i = 1; i < p.size(); i++) { // 如果上一个交点在向量右侧则弹出队尾, 第一个交点在向量右侧则弹出队首
198         for ( ; t.size() && h[p[i]].P_pos(t.back()) >= 0; q.pop_back(), t.pop_back());
199         for ( ; t.size() && h[p[i]].P_pos(t[0]) >= 0; q.pop_front(), t.pop_front());
200         if (h[p[i]].L_pos(h[q.back()]) != 0) t.emplace_back(h[q.back()].inter(h[p[i]])),
201         q.emplace_back(p[i]);
202     }
203     else if (h[p[i]].P_pos(h[q.back()]).a > 0) { // 如果极角相同, 留最靠左的
204         q.back() = p[i];
205         if (t.size()) t.back() = h[q[q.size() - 2]].L_inter(h[q.back()]);
206     }
207 }
208
209 for ( ; t.size() && h[q[0]].P_pos(t.back()) >= 0; q.pop_back(), t.pop_back());
210 t.emplace_front(h[q[0]].L_inter(h[q.back()]));
211 return pair{move(q), move(t)}; // 返回线段下标, 交点
212 }
213
214 static auto half_plane_MUL(const vector<line> & h, deque<int> q = {}, deque<point> t = {}) {
215     vector<int> p(h.size()); iota(p.begin(), p.end(), 0); // 下标从 0, 逆时针, 左侧的半平面交, 使用时一定 TMD
216     加上外边框, 最好是矩形的
217     sort(p.begin(), p.end(), [&](int x, int y) { return h[x].v().less(h[y].v()); });
218     q.emplace_back(p[0]);
219     for (int i = 1; i < p.size(); i++) {
220         for ( ; t.size() && h[p[i]].P_pos(t.back()) > 0; q.pop_back(), t.pop_back()) {
221             while (h[q.back()].L_pos(h[q[q.size() - 2]]) == 0) q.pop_back();
222         }
223         for ( ; t.size() && h[p[i]].P_pos(t[0]) > 0; q.pop_front(), t.pop_front()) {
224             while (h[q[0]].L_pos(h[q[1]]) == 0) q.pop_front();
225         }
226         if (h[q.back()].L_pos(h[p[i]]) != 0) t.emplace_back(h[q.back()].L_inter(h[p[i]])),
227         q.emplace_back(p[i]);
228     }
229     else if (h[p[i]].P_pos(h[q.back()]).a == 0) q.emplace_back(p[i]);
230     else if (h[p[i]].P_pos(h[q.back()]).a > 0) {
231         while (q.size() >= 2 && h[q.back()].L_pos(h[q[q.size() - 2]]) == 0) q.pop_back();
232         q.back() = p[i];
233         if (t.size()) t.back() = h[q[q.size() - 2]].L_inter(h[q.back()]);
234     }
235 }
236
237 for ( ; t.size() && h[q[0]].P_pos(t.back()) > 0; q.pop_back(), t.pop_back()) {
238     while (h[q.back()].L_pos(h[q[q.size() - 2]]) == 0) q.pop_back();
239 }
240 t.emplace_front(h[q[0]].L_inter(h[q.back()]));
241 return pair{move(q), move(t)}; // 返回的允许重点, 重边, 无用边经过点
242 }
243
244 static auto min_circle_over(vector<point> p) { // 使用的半径是正常半径
245     shuffle(p.begin(), p.end(), rng); circle o(p[0], 0);
246     for (int i = 0; i < p.size(); i++) if (o.r < (o.o - p[i]).len()) {
247         o = {p[i], 0};
248         for (int j = 0; j < i; j++) if (o.r < (o.o - p[j]).len()) {
249             o = {(p[i] + p[j]) / 2, (p[i] - p[j]).len() / 2};
250             for (int k = 0; k < j; k++) if (o.r < (o.o - p[k]).len()) {
251                 if (line(p[i], p[j]).P_pos(p[k]) != 0) {
252                     o = circle(p[i], p[j], p[k]);
253                 } else if ((p[i] - p[k]).len() < (p[j] - p[k]).len()) {
254                     o = {(p[j] + p[k]) / 2, (p[j] - p[k]).len() / 2};
255                 } else {
256                     o = {(p[i] + p[k]) / 2, (p[i] - p[k]).len() / 2};
257                 }
258             }
259         }
260     }
261 }

```

```

247         }
248     }
249 }
250 }
251 return o;
252 }
253
254 static auto min_mat_over(const vector<point> & p, T res = 4e18, array<point, 4> o = {}) { // 返回覆盖面积
    的俩倍
255     for (int i = 0, j = p.size() - 1, k = 1, l = 0, r = 0; i < p.size(); j = i++) {
256         line now = line(p[j], p[i]);
257         for ( ; abs(now.P_pos(p[k])) <= abs(now.P_pos(p[(k + 1) % p.size()])); (k += 1) %= p.size());
258         for ( ; now.P_dir(p[r]) <= now.P_dir(p[(r + 1) % p.size()]); (r += 1) %= p.size());
259         if (!i) l = r;
260         for ( ; now.P_dir(p[l]) >= now.P_dir(p[(l + 1) % p.size()]); (l += 1) %= p.size());
261         auto tmp = abs(now.P_pos(p[k])) * (now.P_dir(p[r]) - now.P_dir(p[l])) / now.v().square();
262         if (tmp < res) {
263             res = tmp;
264             o[0] = l == j ? p[l] : now.P_proj(p[l]), o[1] = r == i ? p[r] : now.P_proj(p[r]);
265             auto tt = now.v().rotat();
266             tt = tt * now.P_dis_L(p[k]) / tt.len();
267             o[2] = o[1] + tt, o[3] = o[0] + tt;
268         }
269     }
270     return pair{res, o};
271 }
272 };

```

三维几何基础:

图形的记录:

平面: 利用平面上一点 $P_0(x_0, y_0, z_0)$ 和 its 法向量 $\mathbf{n}$ 来表示, $\forall$ 平面上点 $P(x, y, z), \mathbf{n} \cdot \overrightarrow{PP_0} = 0$

$\Rightarrow A(x - x_0) + B(y - y_0) + C(z - z_0) = 0$, 即 $Ax + By + Cz + D = 0, D = -(Ax_0 + By_0 + Cz_0)$ 为一般式

基本操作:

直线、平面之间的夹角:

```

1  template <class type>
2  struct point : array<type, 3> {
3      using p = array<type, 3>;
4      type operator / (const point & t) const { return p::at(0) * t[0] + p::at(1) * t[1] + p::at(2) * t[2]; }
5      point operator * (const point & t) const { return {p::at(1) * t[2] - p::at(2) * t[1], p::at(2) * t[0] -
    p::at(0) * t[2], p::at(0) * t[1] - p::at(1) * t[0]}; }
6      point operator + (const point & t) const { return point(*this) += t; }
7      point operator - (const point & t) const { return point(*this) -= t; }
8      point operator * (const type & t) const { return point(*this) *= t; }
9      point operator / (const type & t) const { return point(*this) /= t; }
10     point & operator += (const point & t) { return *this = {p::at(0) + t[0], p::at(1) + t[1], p::at(2) +
    t[2]}; }
11     point & operator -= (const point & t) { return *this = {p::at(0) - t[0], p::at(1) - t[1], p::at(2) -
    t[2]}; }
12     point & operator *= (const type & t) { return *this = {p::at(0) * t, p::at(1) * t, p::at(2) * t}; }
13     point & operator /= (const type & t) { return *this = {p::at(0) / t, p::at(1) / t, p::at(2) / t}; }
14     type sqrlen() const { return *this / *this; }
15     type len() const { return sqrt(sqrlen()); }
16
17     void shake(type eps = 1e-10) { // 微小扰动, 去掉四点共面
18         uniform_real_distribution<type> dist(-0.5, 0.5);
19         for (auto & t : *this) {
20             t += dist(rd) * eps;
21         }
22     }
23
24     struct line : array<point, 2> {
25         using p = array<point, 2>; // 0 -> 1
26         point v() { return p::at(1) - p::at(0); }
27         point pos(const point & t) { return v() * (t - p::at(0)); } // sqrlen() == 0 在直线上
28         point pos(const line & t) { return v() * t.v(); } // sqrlen() == 0 平行
29         type dir(const line & t) { return v() / t.v(); } // == 0 垂直, > 0 同向

```

```

30     type dis_to_line(const point & t) { return pos(t).len() / v().len(); }
31     bool equal(const line & t) { return dir(t) > 0 && pos(t[0]).sqrLen() == 0; }
32     point intersect(const line & t) { // 需要保证不平行, 如果直线异面, 返回的是俩直线的垂线在 *this 直线上的垂足
33         return p::at(0) + v() * (t.pos(p::at(0)) / pos(t)) / pos(t).sqrLen();
34     }
35 };
36
37 struct face : array<point, 3> {
38     using p = array<point, 3>; // 从上方看逆时针 0 -> 1 -> 2
39     point v() { return (p::at(1) - p::at(0)) * (p::at(2) - p::at(0)); } // .len() = 2 * 面积
40     type pos(const point & t) { return v() / (t - p::at(0)); } // > 0 严格在上方, == 0 在面上
41     type pos(const line & t) { return v() / t.v(); } // == 0 平行
42     point pos(const face & t) { return v() * t.v(); } // sqrLen() == 0 平行
43     point dir(const line & t) { return v() * t.v(); } // sqrLen() == 0 垂直
44     type dir(const face & t) { return v() / t.v(); } // == 0 垂直, > 0 同向
45     type dis_to_face(const point & t) { return pos(t) / v().len(); } // 在法向量上的有向投影
46     bool equal(const face & t) { return dir(t) > 0 && pos(t).len() == 0 && pos(t[0]) == 0; } // 面是否相
    等
47     type volume(const point & t) { return (p::at(0) - t) * (p::at(1) - t) / (p::at(2) - t); } // 混合积
    = 6 * 有向体积
48     point centroid(const point & t) { return p::at(0) + p::at(1) + p::at(2) + t; } // 4 * 三棱锥重心
49 };
50
51 struct hull : vector<face> {
52     using f = vector<face>;
53     hull(vector<point> t = {}) {
54         if (t.size() < 4) return; // 不允许四点共面
55         shuffle(t.begin(), t.end(), rd, f::push_back({t[0], t[1], t[2]}), f::push_back({t[2], t[1],
    t[0]}));
56         for (int i = 3; i < t.size(); i++) {
57             hull tmp;
58             set<pair<point, point>> edge;
59             for (auto & x : *this) {
60                 if (x.pos(t[i]) < 0) tmp.emplace_back(x);
61                 else edge.emplace(x[0], x[1]), edge.emplace(x[1], x[2]), edge.emplace(x[2], x[0]);
62             }
63             for (auto & [x, y] : edge) {
64                 if (!edge.count({y, x})) tmp.push_back({x, y, t[i]});
65             }
66             swap(*this, tmp);
67         }
68     }
69     template <class ret = type> // 2 * 表面积
70     ret area(ret t = 0) { return accumulate(f::begin(), f::end(), t, [](ret t, auto & x) { return t +
    x.v().len(); }); }
71     template <class ret = type> // 6 * 体积
72     ret volume(ret t = 0) { return accumulate(f::begin(), f::end(), t, [](ret t, auto & x) { return t +
    x.volume({0, 0, 0}); }); }
73     point centroid(point G = {}, type sum = 0) { // 重心
74         for (auto & x : *this) {
75             sum += x.volume({0, 0, 0}), G += x.centroid({0, 0, 0}) * x.volume({0, 0, 0});
76         }
77         return G / 4 / sum;
78     }
79 };
80 };

```

Pick定理

Pick定理: 给定顶点均为整点的简单多边形, 其面积 A , 内部格点数 i , 边上格点数 b 的关系为 $A = i + \frac{b}{2} - 1$

凸包

多边形: 由在同一平面且不再同一直线上的多条线段首尾顺次连接且不相交所组成的图形叫多边形

简单多边形: 简单多边形是除相邻边外其它边不相交的多边形

凸多边形：凸多边形是指内角大小都在 $[0, \pi]$ 范围内的简单多边形,任意凸多边形外角和 2π ,内角和 $(n - 2)\pi$

凸包：在平面内能包含所有给定点的最小凸多边形叫做凸包 $\Leftrightarrow \forall$ 给定集合 X ,所有包含 X 的凸集的交集 S 为 X 的凸包

凸包用最小的周长围住了给定的所有点

二维凸包：

Andrew求凸包：时间复杂度： $O(n \log n)$,瓶颈在于排序

将所有点以横坐标为第一关键字,纵坐标为第二关键字排序,排序后最小的元素和最大的元素一定在凸包上

因为是凸多边形,我们如果从一个点出发逆时针走,轨迹总是「左拐」的,一旦出现右拐,就说明这一段不在凸包上

因此可以用一个单调栈来维护上下凸壳

因为从左向右看,上下凸壳所旋转的方向不同,为了让单调栈起作用,我们首先升序枚举求出下凸壳,然后降序求出上凸壳

求凸壳时,发现即将进栈的点 P 和栈顶的两个点 $(S_1, S_2, S_1$ 为栈顶)行进的方向向右旋转,即叉积小于0： $\overrightarrow{S_1 S_2} \times \overrightarrow{S_1 P} > 0$

则弹出栈顶,回到上一步继续检测,直到 $\overrightarrow{S_1 S_2} \times \overrightarrow{S_1 P} \leq 0$ 或者栈内仅剩一个元素为止

通常情况下不需要保留位于凸包边上的点,因此上面一段中 $\overrightarrow{S_1 S_2} \times \overrightarrow{S_1 P} > 0$ 这个条件中的「 $>$ 」可以视情况改为 $\geq$

三维凸包：时间复杂度 $O(n^2)$

(i) 首先对其微小扰动,避免出现四点共面的情况

(ii)

对于一个已知凸包,新增一个点 P ,将 P 视作一个点光源,向凸包内做射线,可以知道,光线的可见面和不可见面一定是由若干条棱隔开的

(iii)将光的可见面删去,并新增由其分割棱与 P 构成的平面,重复此过程即可

半平面交

最小圆覆盖

旋转卡壳

三角剖分

平面最近点对

划分左右区域,先得出左右区域的最小值 d

对于最近点对在中线左右两侧,考虑对在中线附近的横坐标 x 距离不超过按照 d 的点 y 进行排序

然后枚举所有点找出纵坐标 y 距离不超过 d 的点暴力计算,可以证明不超过6个点

对 y 排序过程可以用归并合并并在分治中,总复杂度 $O(\log n)$

```
1  vector<point> a(n + 1);
2
3  for (int i = 1; i <= n; i++) {
4      cin >> a[i][0] >> a[i][1];
5  }
6
7  sort(a.begin() + 1, a.end());
8
9  auto dfs = [&](auto dfs, int l, int r) -> int {
```

```

10     if (l >= r) {
11         return 4e18;
12     }
13
14     int mid = l + r >> 1;
15     int mid_x = a[mid][0]; //注意先取出来后再分治,因为分治后因为归并排序出来不是原数组
16     int d = min(dfs(dfs, l, mid), dfs(dfs, mid + 1, r));
17
18     vector<point> f(a.begin() + l, a.begin() + mid + 1), g(a.begin() + mid + 1, a.begin() + r + 1);
19
20     for (int i = 0, j = 0, p = l; i < f.size() || j < g.size(); ) {
21         if (i < f.size() && (j >= g.size() || f[i][1] < g[j][1])) {
22             a[p++] = f[i++];
23         } else {
24             a[p++] = g[j++];
25         }
26     }
27
28     vector<point> tmp;
29     for (int i = l; i <= r; i++) {
30         if ((a[i][0] - mid_x) * (a[i][0] - mid_x) < d) {
31             tmp.push_back(a[i]);
32         }
33     }
34
35     for (int i = 0; i < tmp.size(); i++) {
36         for (int j = i + 1; j < tmp.size() && (tmp[j][1] - tmp[i][1]) * (tmp[j][1] - tmp[i][1]) < d; j++) {
37             d = min(d, (tmp[i] - tmp[j]) & (tmp[i] - tmp[j]));
38         }
39     }
40
41     return d;
42 };

```